



Trabajo Práctico 1

Arquitectura de Software 1er cuatrimestre de 2026

Integrantes:

- Abramovich, Martín - 108762 - mabramovich@fi.uba.ar
- Giordano Hoyo, Abril - 108760 - agiordano@fi.uba.ar
- Laudadio, Fernando - 93574 - flaudadio@fi.uba.ar
- Taras, Julieta Belen - 104728 - jtaras@fi.uba.ar

Índice

Índice.....	2
1. Introducción.....	3
1. Análisis Inicial.....	3
2.1. Arquitectura, Código e Infraestructura.....	4
2.2. Atributos de Calidad Clave.....	4
2.2.1. Disponibilidad y Fiabilidad.....	5
2.2.2. Performance.....	5
2.2.3. Escalabilidad.....	5
2.3. Análisis de Atributos de Calidad.....	5
2.3.1. Disponibilidad.....	6
2.3.2. Performance.....	6
2.3.3. Escalabilidad.....	6
2.3.4. Fiabilidad.....	6
2.3.5. Visibilidad.....	6
2.3.6. Testeabilidad.....	7
2.3.7. Portabilidad.....	7
2.3.8. Interoperabilidad.....	7
2.3.9. Manejabilidad.....	7
2.3.10. Seguridad.....	7
2.3.11. Simplicidad.....	8
2.3.12. Modificabilidad.....	8
2.4. Comportamiento del Servicio.....	8
2.4.1. Servicio GET /accounts.....	8
2.4.2. Servicio GET /rates.....	10
2.4.3. Servicio GET /log.....	12
2.4.4. Servicio PUT /rates.....	13
2.4.5. Servicio PUT /accounts/:id/balance.....	15
2.4.6. Servicio POST /exchange.....	17
2.5. Components & Connectors.....	19
2. Métricas de Negocio.....	19
3. Tácticas Aplicadas.....	21
4.1. Táctica 1: Transactions.....	21
4.2. Táctica 2: Limit Access.....	23
4.3. Táctica 3, 4 y 5: Distributed State Cache con Scale out not up e Introduce Concurrency.....	26
4. Conclusiones.....	31

1. Introducción

arVault es una fintech que opera una billetera virtual orientada al cambio de monedas. Su objetivo es ofrecer la tasa de cambio más conveniente del mercado.

El servicio fue construido rápidamente por su fundador bajo la premisa de "fake it until you make it": priorizar funcionalidad por sobre solidez arquitectónica para atraer inversores. Esto resultó en un backend funcional pero con decisiones de diseño que comprometen su comportamiento bajo carga real.

Tras el crecimiento en usuarios, comenzaron a aparecer reclamos relacionados con el servicio de cambio de monedas. Frente a esto, arVault contrató a un equipo de arquitectos para analizar el sistema, identificar los problemas y proponer mejoras concretas.

Este informe documenta ese trabajo: se analiza la arquitectura, el código y la infraestructura recibidos; se identifican y justifican los atributos de calidad clave; se realizan pruebas de carga para caracterizar el comportamiento actual; y se proponen e implementan tácticas de mejora, midiendo su impacto cuando es posible y justificando los trade-offs en todos los casos.

El sistema recibido consiste en una aplicación Node.js con Express que expone una API REST para consultar tasas de cambio, administrar cuentas y ejecutar operaciones de intercambio de moneda. Está expuesta a través de un proxy inverso Nginx y desplegada mediante Docker Compose en un único servidor. El almacenamiento de datos se realiza sobre archivos .json en disco. La infraestructura de observabilidad incluye StatsD, Graphite, Grafana y cAdvisor para la recolección y visualización de métricas, y Artillery como herramienta de generación de carga.

Si bien la aplicación cumple con las funcionalidades básicas de un servicio de exchange, presenta limitaciones arquitectónicas que impactan directamente en los atributos de calidad más críticos para un sistema financiero. El análisis y la resolución de estas limitaciones es el eje central de este trabajo.

1. Análisis Inicial

Realizar un diagnóstico integral de arVault exige analizar su arquitectura técnica en función de los objetivos estratégicos de la compañía. Por ello, además de describir la infraestructura y el código, es fundamental evaluar cómo estas definiciones impactan tanto en los requerimientos funcionales como en los atributos de calidad críticos para el éxito de una billetera digital.

2.1. Arquitectura, Código e Infraestructura

Analizando su infraestructura, el sistema corre íntegramente en un único servidor mediante Docker Compose. Los servicios que lo componen son:

- Nginx: actúa como reverse proxy, recibiendo el tráfico entrante en el puerto 5555 y redirigiéndolo al servicio de API.
- API (Node.js + Express): servicio principal que expone los endpoints de cambio de monedas. Corre como una única instancia sin replicación.
- Graphite + StatsD: reciben y almacenan métricas de tiempo de serie.
- cAdvisor: recolecta métricas de recursos de los containers (CPU, memoria) y las envía a Graphite vía StatsD.
- Grafana: visualización de métricas en tiempo real.

No hay ningún mecanismo de escalado horizontal ni balanceo de carga real: el upstream de Nginx apunta a una única instancia fija del servicio.

Si observamos el código fuente base, el servicio provisto por la aplicación expone cuatro grupos de endpoints: cuentas (/accounts), tasas (/rates), cambio (/exchange) y logs (/log). La lógica de negocio está separada entre app.js (routing), exchange.js (lógica) y state.js (persistencia).

El endpoint más crítico es POST /exchange, que ejecuta hasta tres llamadas secuenciales al servicio de transferencias (transfer()), cada una con una latencia simulada de entre 200 y 400 ms. Esto implica que cada operación de cambio exitosa tarda entre 400 ms y 800 ms solo en transferencias, llegando potencialmente a ~1200 ms en el peor caso (cuando una transferencia falla y hay que revertirla).

No existe ningún mecanismo de control de concurrencia sobre los saldos: si dos requests modifican el mismo saldo simultáneamente, se producen race conditions que pueden generar inconsistencias.

Si ponemos el foco en la persistencia, podemos mencionar que el estado del sistema (cuentas, tasas y log de transacciones) se guarda en tres archivos .json mediante setInterval. Las cuentas y el log se persisten cada 1 segundo, y las tasas cada 5 segundos. Esto implica que ante una caída del servicio se pueden perder hasta 5 segundos de transacciones. Además, las escrituras no son atómicas, lo que podría corromper un archivo si el proceso se interrumpe durante el guardado.

2.2. Atributos de Calidad Clave

Dado que es imposible optimizar simultáneamente todos los atributos de calidad, la arquitectura exige priorizar aquellos alineados con los objetivos de negocio. Se enumeran a continuación los atributos de calidad que consideramos críticos para la aplicación analizada.

2.2.1. Disponibilidad y Fiabilidad

En el contexto de una billetera digital, la disponibilidad es un atributo primario porque el sistema debe estar operativo y listo para llevar adelante sus tareas cuando el usuario lo requiere. Cualquier período de inactividad se traduce directamente en una pérdida de confianza del cliente y reputación, lo cual es crítico para una startup que busca captar inversiones.

A su vez, la fiabilidad se vuelve esencial como la visión interna de la disponibilidad, ya que garantiza que el sistema continúe operando correctamente incluso ante fallas parciales. En un servicio de cambio de moneda, es vital que una falla en un componente menor no detenga la capacidad del usuario de acceder a sus fondos o completar una transacción.

2.2.2. Performance

Para arVault, este atributo es clave debido a su objetivo de negocio diferencial: ofrecer la tasa de cambio más conveniente del mercado. La rapidez en el tiempo de respuesta (Response Time) y el tiempo de completitud de la transacción (Completion Time) son factores determinantes para la experiencia del usuario. Si el procesamiento es lento, se pierde la ventaja competitiva de la tasa en tiempo real, lo que genera reclamos y críticas negativas que expulsan a potenciales inversores. Superar a la competencia en términos de velocidad de procesamiento es, por lo tanto, un requerimiento de calidad prioritario.

2.2.3. Escalabilidad

Dado que el plan de negocio de arVault contempla captar una "buena base de usuarios" y expandir su presencia en el mercado tras obtener más inversiones, la escalabilidad es también un atributo fundamental. Un sistema que no sea capaz de escalar para soportar un número creciente de transacciones y usuarios concurrentes corre el riesgo de degradar su performance y, en casos extremos, perder su disponibilidad. Para una startup en fase de crecimiento, la arquitectura debe permitir este aumento de capacidad sin requerir un esfuerzo desproporcionado ni interrupciones en el servicio.

2.3. Análisis de Atributos de Calidad

Tras identificar los atributos críticos para el negocio, es necesario evaluar las propiedades restantes del sistema bajo una visión integral. Este enfoque permite gestionar los trade-offs arquitectónicos, reconociendo que favorecer un atributo suele impactar negativamente en otros.

2.3.1. Disponibilidad

Actualmente la disponibilidad es crítica y muy pobre. El desarrollador optó por una instancia única de la API sin replicación ni mecanismos de health checks, lo que significa que si el proceso falla, el servicio cae totalmente. A su vez, al no realizar operaciones atómicas, la integridad se ve altamente comprometida.

2.3.2. Performance

El diseño utiliza operaciones bloqueantes secuenciales en el endpoint /exchange, donde cada llamado a transfer() añade latencia, alcanzando hasta 1200 ms en el peor caso. Además, la persistencia genera un I/O bloqueante sobre archivos JSON, las búsquedas en memoria son de orden lineal $O(n)$ y el endpoint de /log cuya respuesta puede crecer infinitamente no soporta paginación.

2.3.3. Escalabilidad

El diseño hace que la escalabilidad horizontal sea arquitectónicamente imposible en su estado actual. El estado completo del sistema (cuentas, tasas, log) vive en memoria RAM del proceso y se sincroniza periódicamente a archivos locales en disco. Si se levantan dos instancias, cada una mantendría su propia copia en memoria independiente. Las modificaciones de saldo realizadas en la instancia A no serían visibles para la instancia B, y ambas sobrescribirían los mismos archivos en disco generando corrupción de datos.

2.3.4. Fiabilidad

La fiabilidad se ve comprometida principalmente por dos problemas: la ventana de pérdida de datos y la ausencia de transacciones atómicas. La decisión de usar guardado periódico (cada 1 a 5 segundos) genera ventanas donde, ante un fallo, se pierde información financiera confirmada. Además, el sistema carece de transacciones ACID, por lo que un crash entre actualizaciones de saldos deja al sistema en un estado inconsistente, situación altamente crítica en operaciones en donde se ven involucrados activos económicos.

2.3.5. Visibilidad

La presencia de Nginx como punto de entrada único facilita el registro centralizado de todas las solicitudes. Las herramientas cAdvisor, StatsD, Graphite y Grafana proveen métricas de infraestructura (CPU, memoria, red) de los containers en tiempo real con persistencia. Sin embargo, a nivel de la aplicación en sí el panorama es diferente. Las métricas de negocio, como el

volumen operado por moneda, el tiempo de procesamiento de cada exchange o la tasa de operaciones fallidas, son completamente invisibles.

2.3.6. Testeabilidad

El código no incluye ningún test unitario, de integración ni de contrato. Más allá de la ausencia de tests, el diseño dificulta estructuralmente agregarlos ya que el código presenta un acoplamiento fuerte entre módulos (app depende de exchange y este de state). No se observan pipelines de CI/CD que automaticen la ejecución de pruebas ante cambios. El único mecanismo de prueba disponible es la colección de Postman, que requiere intervención manual.

2.3.7. Portabilidad

El uso de Docker y Docker Compose encapsula todas las dependencias del runtime (Node.js, Nginx, Graphite, Grafana) en imágenes reproducibles, eliminando la dependencia del entorno del host. El sistema puede levantarse en cualquier máquina con Docker instalado con un único comando, sin necesidad de configurar manualmente dependencias.

2.3.8. Interoperabilidad

Se considera buena al basarse en una API REST estándar que utiliza HTTP y JSON, lo que facilita la comunicación con diversos clientes. Aunque posee endpoints semánticos, el sistema es deficiente para integrarse en entornos distribuidos que exigen coordinación y consistencia. Esta debilidad se agrava por la falta de versionado y de documentación formal.

2.3.9. Manejabilidad

Docker Compose simplifica el despliegue inicial, pero la operación es difícil: cambiar cualquier configuración requiere reconstruir la imagen. Si bien cuenta con un buen nivel en cuanto al monitoreo en tiempo real del sistema, en cuanto a debugging el mismo se ve afectado por el manejo de errores casi inexistente y con mensajes poco informativos.

2.3.10. Seguridad

El servicio no maneja autenticación interna, sino que la delega a un componente externo (vaultSec) que no está replicado. No hay autorización: cualquier cliente podría teóricamente modificar saldos de otros. Tampoco se implementa rate limiting a nivel de aplicación (Nginx tampoco lo configura), lo que deja al servicio expuesto a ataques de fuerza bruta o denegación de servicio a nivel de aplicación. La ausencia de HTTPS en la configuración de Nginx implica que el tráfico entre clientes y el servidor viaja sin cifrar.

2.3.11. Simplicidad

La arquitectura es minimalista, con solo tres archivos principales y una lógica de negocio directa, evitando el *over-engineering*. Esta simplicidad tiene valor real: el código es fácil de leer y el flujo de una operación es rastreable de extremo a extremo sin necesidad de navegar múltiples capas de abstracción. El costo de esa simplicidad, sin embargo, es que muchas de las limitaciones analizadas en otros atributos son consecuencia directa de haber sacrificado estructura a favor de velocidad de desarrollo.

2.3.12. Modificabilidad

Se observa una deficiente separación de responsabilidades y un fuerte acoplamiento entre sus módulos. Como reconoce el propio desarrollador, el manejo de saldos de cuentas está indebidamente integrado, lo que dificulta su capacidad de evolución y genera deuda técnica. Tampoco existe una capa de repositorio ni de dominio que aisle estas responsabilidades. Agregar soporte para una nueva moneda o cambiar el motor de persistencia requiere modificar múltiples módulos de forma coordinada, incrementando el riesgo de introducir regresiones.

2.4. Comportamiento del Servicio

Con el objetivo de obtener un panorama objetivo del comportamiento del servicio en su estado original, se ejecutaron pruebas de carga y de estrés sobre cada uno de los endpoints expuestos por la API de arVault y se analizaron sus gráficos de *Request State*, *Response Time* y *Resources*.

Para las pruebas de carga se definieron escenarios individuales por endpoint, simulando patrones de consumo representativos de situaciones reales: accesos periódicos a cotizaciones, lecturas frecuentes de cuentas, actualizaciones de saldo y operaciones de cambio, entre otros. Para las pruebas de estrés, en cambio, se utilizó una configuración uniforme para todos los endpoints: un pool de 300 conexiones durante 110 segundos, escalando progresivamente desde 100 hasta 600 req/s.

2.4.1. Servicio GET /accounts

El primer escenario simula la lectura frecuente del listado de cuentas con un pool de 50 conexiones compartidas durante 120 segundos, escalando desde 2 req/s en el calentamiento hasta un pico de 25 req/s y terminando con una fase de recuperación a 5 req/s.

Se procesaron 1.270 requests sin ningún fallo a una tasa promedio de 6 req/s. Los tiempos de respuesta fueron prácticamente despreciables: mínimo de 0 ms, máximo de 4 ms, media de 1,1 ms y p99 de 3 ms. Esto es esperable dado que el endpoint realiza una lectura directa sobre el array en

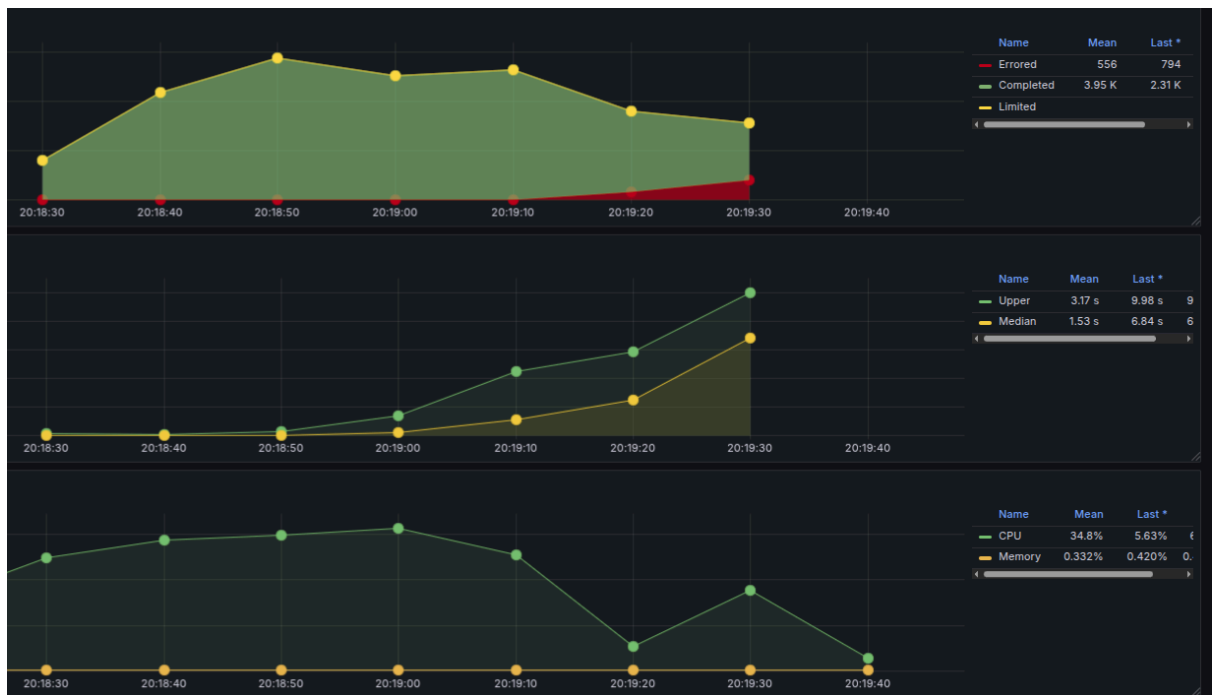
memoria sin ningún procesamiento adicional. El uso de recursos fue igualmente bajo, con un promedio de CPU del 0,975% y memoria del 0,249%.



La prueba de estrés arrojó un promedio de uso de CPU del 33,2% y memoria del 0,332%, valores notablemente superiores a los de la prueba de carga. De los 36.020 requests enviados, 33.764 obtuvieron respuesta exitosa (200), mientras que 2.256 fallaron, distribuidos entre errores de conexión (ECONNRESET: 1.955) y timeouts (ETIMEDOUT: 301), representando una tasa de fallo del 6,3%.

Los tiempos de respuesta mostraron una degradación considerable: la media fue de 1.520,7 ms con una mediana de 135,7 ms, pero el p95 trepó a 7.709,8 ms y el p99 a 9.230,4 ms, evidenciando que una fracción significativa de las solicitudes experimentó latencias extremas.

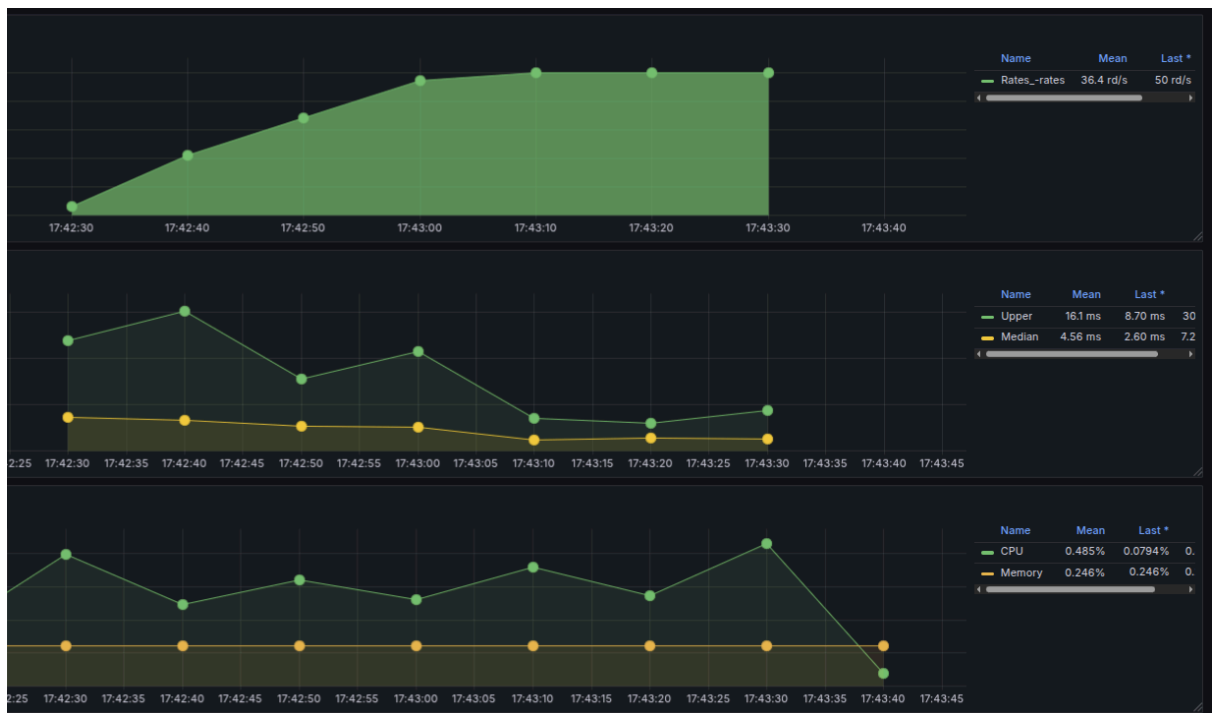
Cabe destacar que la mediana baja debe interpretarse con cautela, ya que puede estar influenciada por respuestas de error generadas rápidamente ante la sobrecarga, y no necesariamente refleja un buen rendimiento del servicio.



2.4.2. Servicio GET /rates

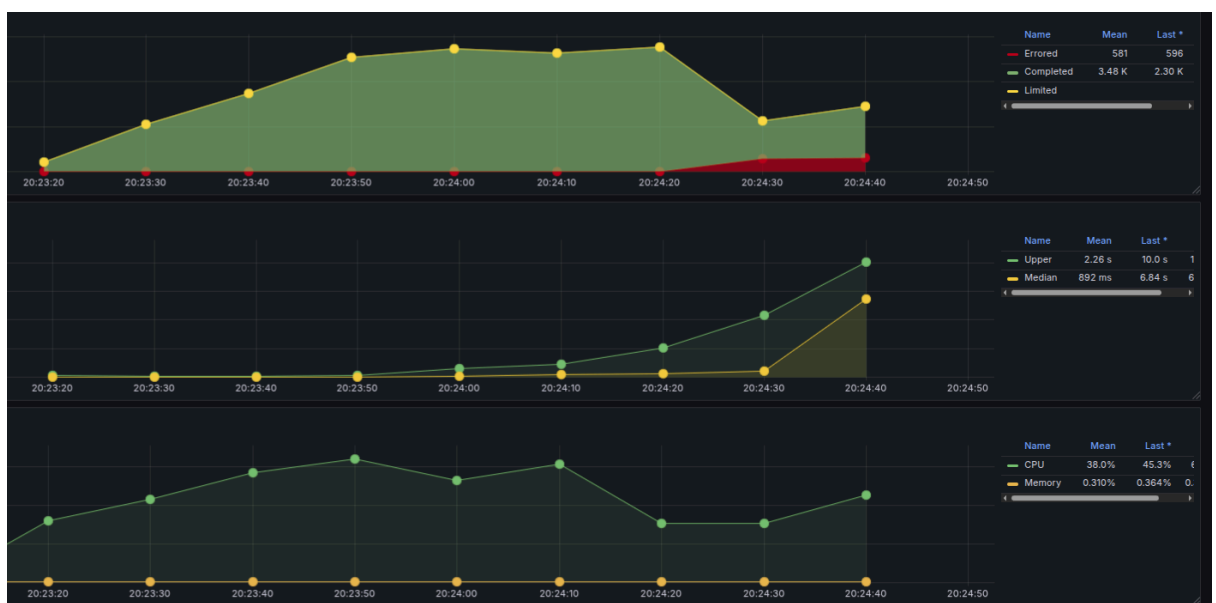
En esta ocasión, la prueba de carga simula el acceso periódico a las cotizaciones vigentes con un pool de 50 conexiones compartidas durante 90 segundos, con una rampa progresiva de 1 a 5 req/s en los primeros 30 segundos y carga sostenida de 5 req/s durante los 60 segundos restantes.

Se procesaron 390 requests sin ningún fallo a una tasa promedio de 3 req/s. Los tiempos de respuesta fueron mínimos: media de 1,4 ms, mediana de 1 ms y p99 de 4 ms. Al igual que GET /accounts, el rendimiento se explica por tratarse de una lectura directa sobre un objeto en memoria. El uso de recursos fue incluso menor que en el endpoint anterior, con un promedio de CPU del 0,485% y memoria del 0,246%.



La prueba de estrés arrojó un CPU del 38% y memoria del 0,310% en promedio, valores similares al caso anterior. De los 36.020 requests, 34.103 fueron exitosos y 1.917 fallaron (ECONNRESET: 1.640, ETIMEDOUT: 277), representando una tasa de fallo del 5,3%, levemente inferior a la de GET /accounts.

Los tiempos de respuesta siguieron el mismo patrón de degradación: media de 1.072,7 ms, mediana de 79,1 ms, p95 de 7.260,8 ms y p99 de 9.047,6 ms, con la misma advertencia respecto a la mediana baja como posible indicador de errores rápidos y no de buen rendimiento.



2.4.3. Servicio GET /log

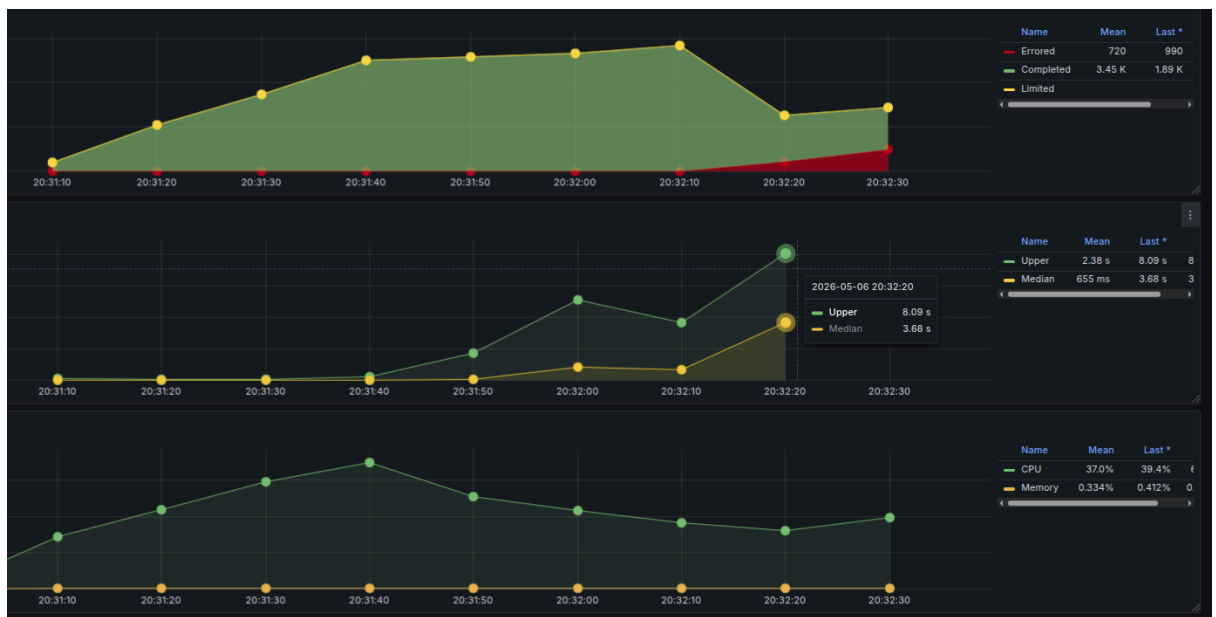
Este escenario simula el acceso al log de transacciones con un pool de 25 conexiones compartidas durante 100 segundos, con carga base de 3 req/s durante 50 segundos, un pico de 12 req/s por 20 segundos representando un momento de troubleshooting, y retorno a 3 req/s por los 30 segundos finales.

Se procesaron 480 requests sin ningún fallo a una tasa promedio de 6 req/s. Los tiempos de respuesta fueron similares a los otros endpoints de lectura: media de 1,2 ms, mediana de 1 ms y p99 de 3 ms. El uso de recursos fue igualmente bajo, con un promedio de CPU del 0,489% y memoria del 0,250%.



La prueba de estrés indica un promedio de CPU del 37% y memoria del 0,334%, en línea con los endpoints anteriores. De los 36.020 requests, 33.723 fueron exitosos y 2.297 fallaron (ECONNRESET: 1.501, ETIMEDOUT: 796), representando una tasa de fallo del 6,4%. Los tiempos de respuesta siguieron el patrón ya observado: media de 1.547,4 ms, mediana de 228,2 ms, p95 de 8.186,6 ms y p99 de 9.416,8 ms.

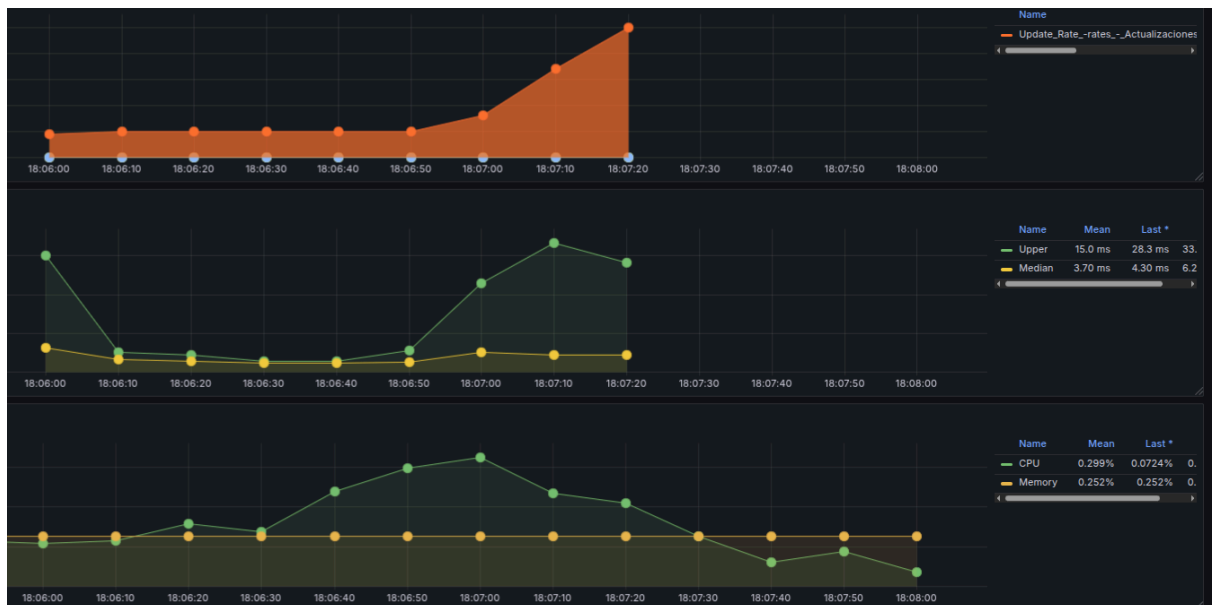
A diferencia de GET /accounts y GET /rates, este endpoint tiene un comportamiento potencialmente degradante a largo plazo: el log crece con cada operación de exchange ejecutada, por lo que su tamaño de respuesta aumentará con el tiempo y podría impactar negativamente tanto en la latencia como en el ancho de banda consumido.



2.4.4. Servicio PUT /rates

El escenario de carga actual simula la actualización de tasas de cambio con un pool de 20 conexiones compartidas durante 120 segundos, con actualizaciones intermitentes a 1 req/s por 60 segundos, una escalada progresiva de 1 a 6 req/s durante 30 segundos simulando volatilidad de mercado, y una fase de estabilización a 2 req/s por los 30 segundos finales.

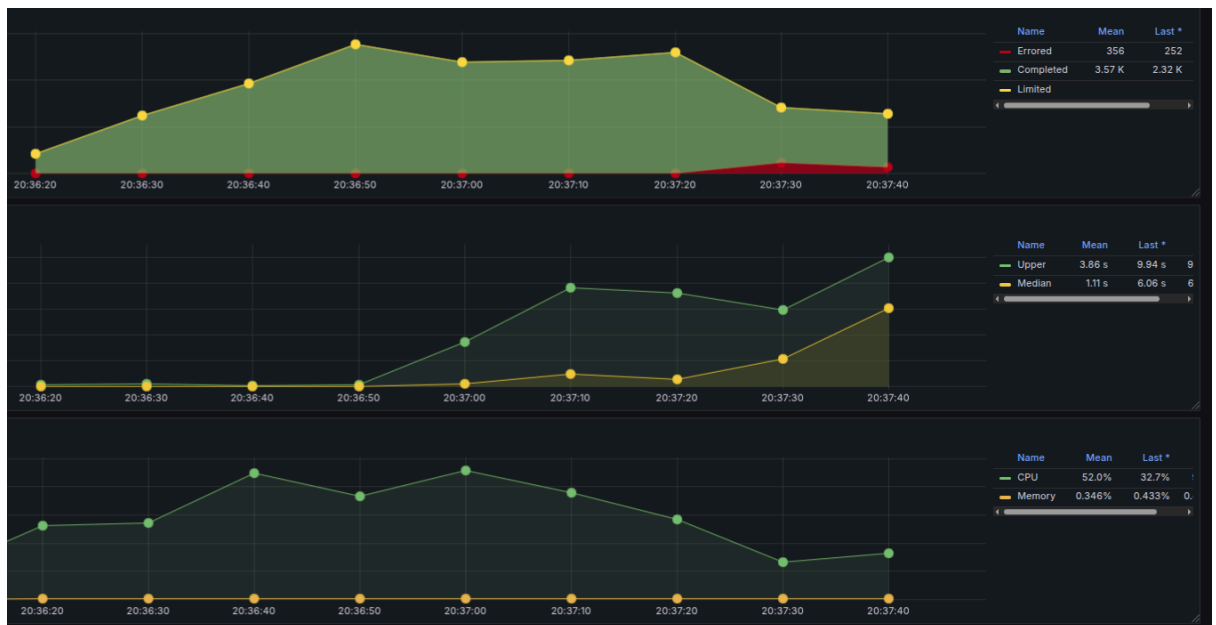
Se procesaron 225 requests sin ningún fallo a una tasa promedio de 2 req/s. Los tiempos de respuesta fueron mínimos: media de 1,7 ms, mediana de 1 ms y p99 de 5 ms, incluso durante el pico de volatilidad. Esto es consistente con la naturaleza de la operación, que se limita a actualizar dos entradas en memoria sin persistencia sincrónica. El uso de recursos fue bajo, registrando un promedio de CPU del 0,299% y memoria del 0,353%.



En este caso, la prueba de estrés muestra un CPU del 52% y memoria del 0,346% en promedio, siendo el valor de CPU más elevado registrado hasta el momento entre los endpoints de lectura y actualización simple. De los 36.020 requests, 34.816 fueron exitosos y 1.204 fallaron (ECONNRESET: 1.164, ETIMEDOUT: 40), representando la tasa de fallo más baja observada hasta ahora, del 3,3%.

Los tiempos de respuesta mostraron una degradación moderada respecto a los casos anteriores: media de 1.152,6 ms, mediana de 210,6 ms, p95 de 6.064,7 ms y p99 de 8.024,5 ms.

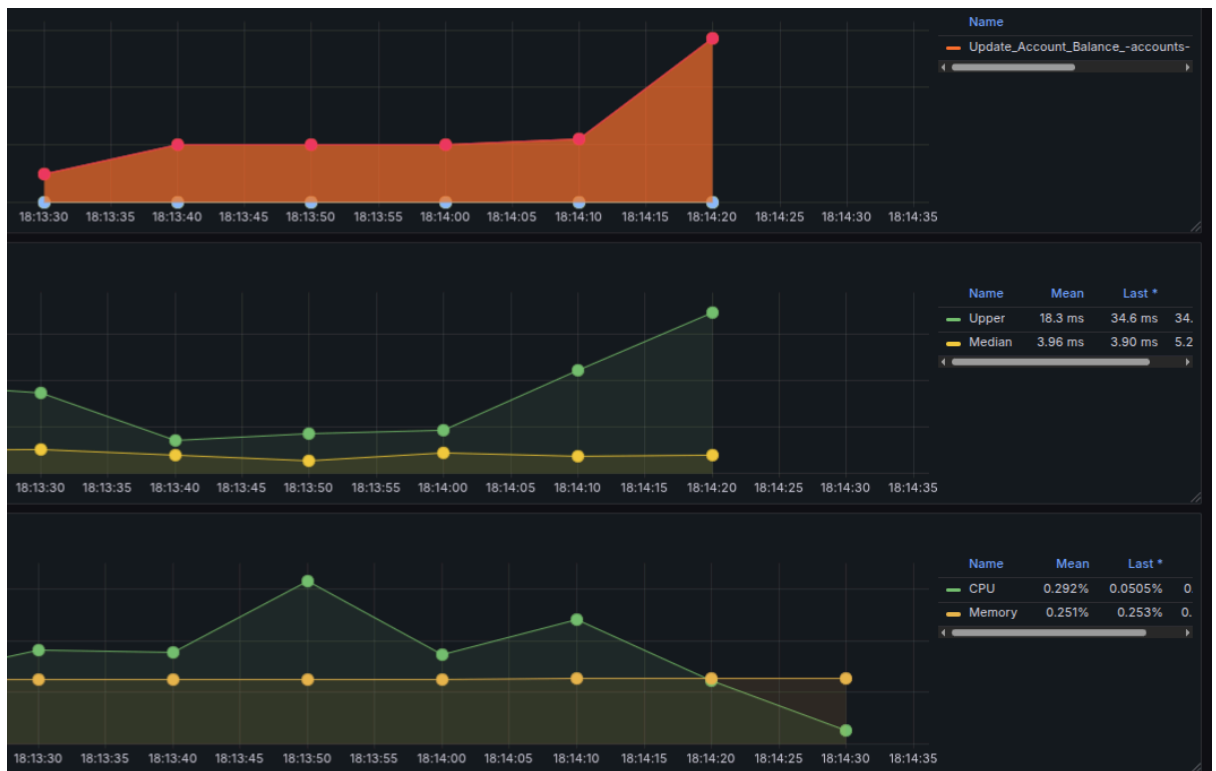
El mayor uso de CPU podría explicarse por la escritura concurrente sobre el objeto de tasas en memoria bajo alta carga, aunque sin impactar significativamente en la tasa de errores.



2.4.5. Servicio PUT /accounts/:id/balance

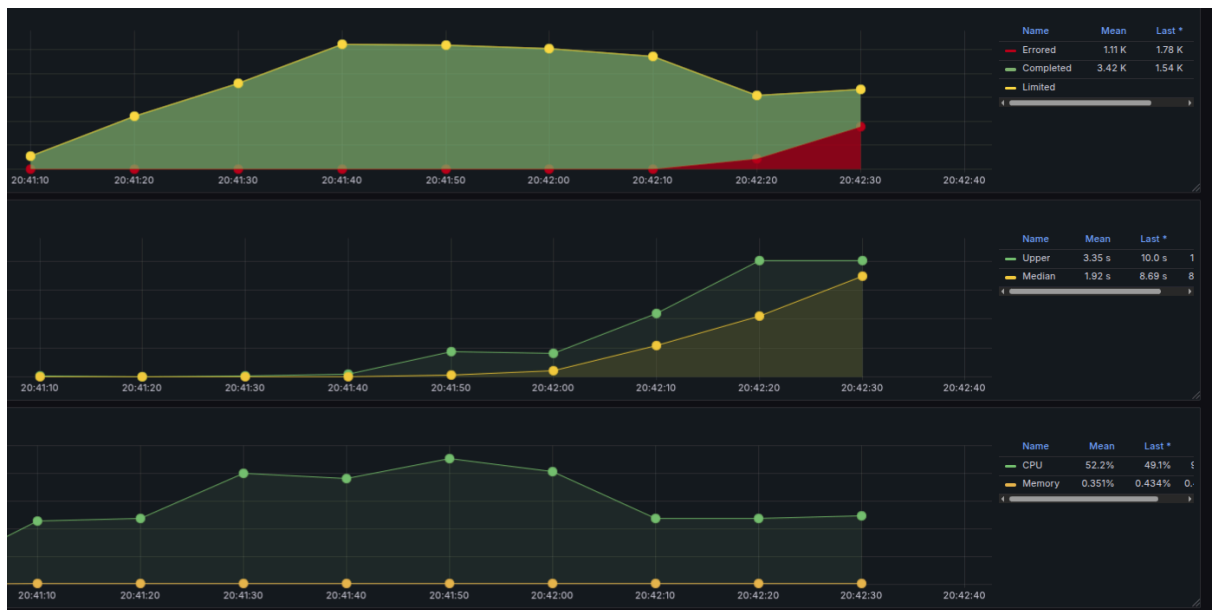
Para este servicio, se simula la actualización de saldos de cuentas con un pool de 30 conexiones compartidas durante 80 segundos, con carga baja y constante a 2 req/s por 40 segundos, una ráfaga de escrituras escalando de 2 a 8 req/s durante 15 segundos, y una fase de enfriamiento a 1 req/s por los 25 segundos finales.

Se procesaron 180 requests sin ningún fallo a una tasa promedio de 2 req/s. Los tiempos de respuesta fueron mínimos: media de 1,4 ms, mediana de 1 ms y p99 de 4 ms, incluso durante la ráfaga de escrituras. La operación es liviana ya que se limita a localizar la cuenta por ID en el array en memoria y actualizar su campo balance sin persistencia sincrónica. El uso de recursos se mantuvo estable, con promedios de CPU del 0,292% y memoria del 0,250%.



La prueba de estrés arrojó un CPU del 52,2% y memoria del 0,351%, valores similares a los de PUT /rates. De los 36.020 requests, 33.025 fueron exitosos y 2.995 fallaron (ECONNRESET: 1.060, ETIMEDOUT: 1.935), representando la tasa de fallo más alta registrada hasta el momento, del 8,3%. A diferencia de PUT /rates, aquí los timeouts predominan sobre los errores de conexión, sugiriendo que el servidor recibe las solicitudes pero no logra procesarlas a tiempo bajo alta concurrencia.

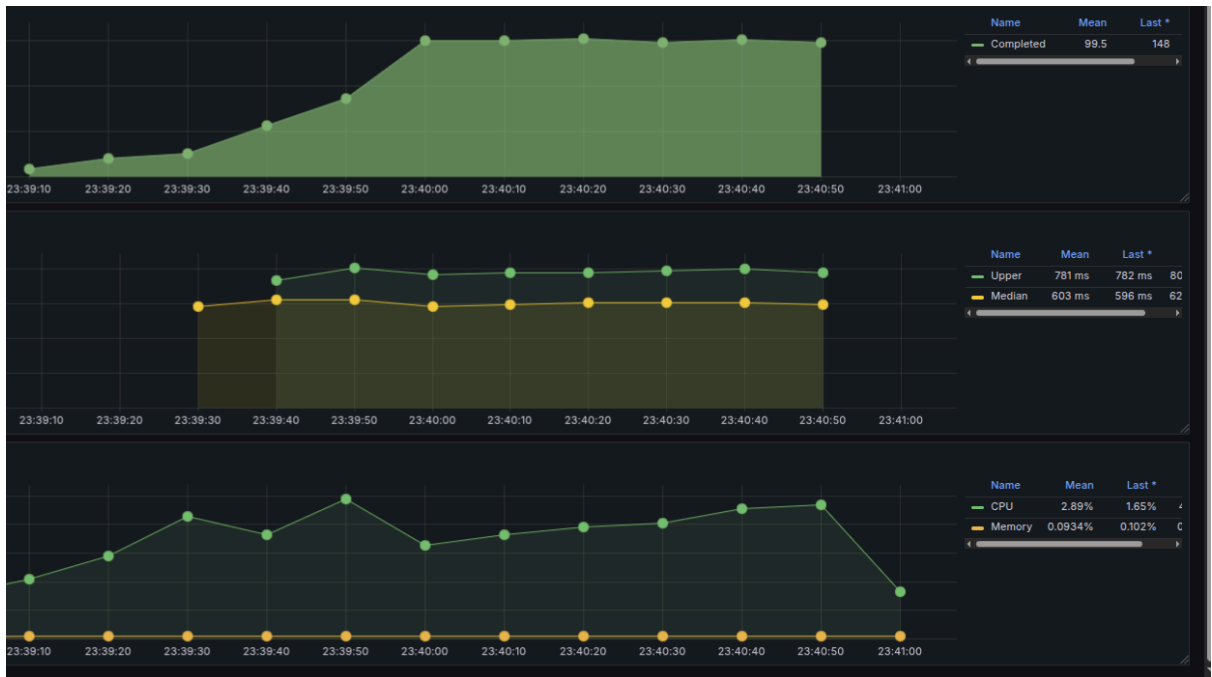
Los tiempos de respuesta fueron los más elevados de todos los endpoints analizados: media de 1.763,3 ms, mediana de 273,2 ms, p95 de 8.692,8 ms y p99 de 9.801,2 ms, evidenciando una degradación considerable bajo condiciones de estrés.



2.4.6. Servicio POST /exchange

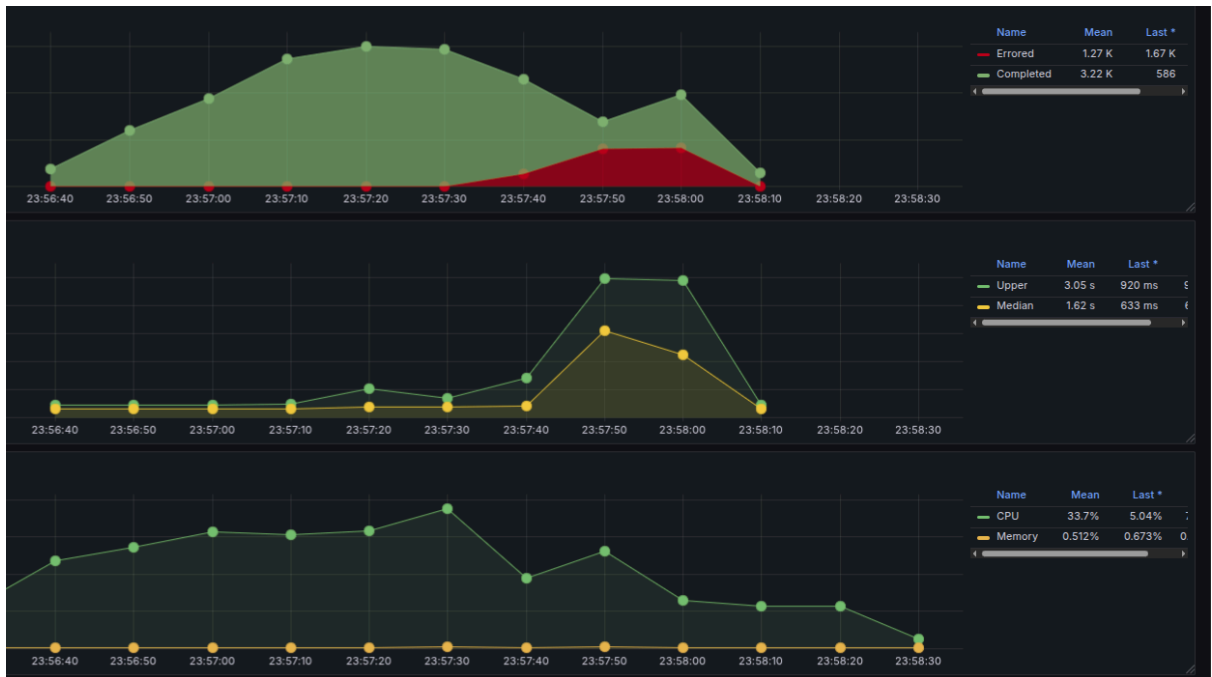
El escenario de carga simula el ciclo diario de operaciones de cambio ARS-USD con un pool de 60 conexiones compartidas durante 170 segundos, modelando distintas franjas horarias: desde 2 req/s en el arranque, escalando hasta un pico de 15 req/s, con declive gradual hasta volver a 2 req/s al final. Los balances de ambas cuentas se inicializan con fondos suficientes para garantizar que ninguna operación falle por falta de saldo (lo mismo se aplica para la prueba de estrés).

Durante la prueba de carga, se procesaron 1.640 requests sin ningún fallo, con una media de 602,1 ms, mediana de 608 ms, p95 de 742,6 ms y p99 de 772,9 ms. Estos valores son consistentes con el diseño del servicio: cada operación realiza dos llamadas secuenciales al servicio de transferencias con demoras de entre 200 y 400 ms cada una, lo que explica que los tiempos se concentren en el rango de 400 a 800 ms. El uso de recursos fue acorde a la carga: CPU promedio de 2,89% y memoria de 0,0934%.



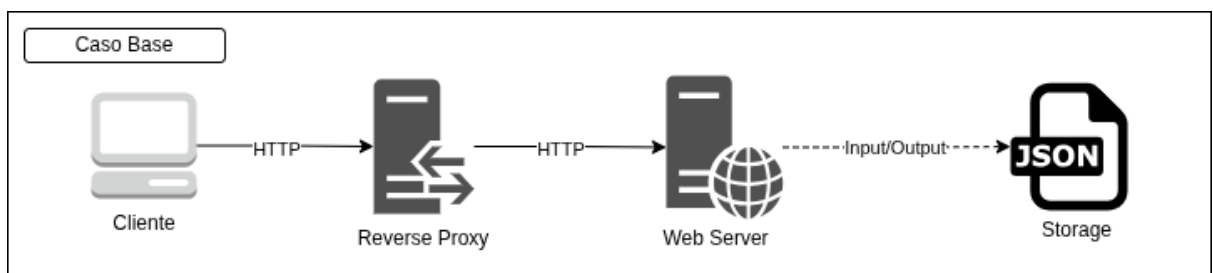
La prueba de estrés expone claramente el punto de quiebre del servicio: de los 36.020 requests generados, 3.803 fallaron (ECONNRESET: 3.801, ETIMEDOUT: 2), representando la tasa de fallo más alta de todos los endpoints analizados, con un 10,6%. Se observan además 6 errores 502, los cuales son particularmente significativos ya que indican que nginx no pudo establecer conexión con Node.js en los momentos de mayor presión, representando una interrupción total del servicio.

Los tiempos de respuesta también fueron los más comprometidos: media de 1.153,6 ms, mediana de 713,5 ms, p95 de 5.272,4 ms y p99 de 7.117 ms. A nivel de recursos, el CPU escaló al 33,7% y la memoria al 0,512%, siendo este último el valor más elevado registrado entre todos los endpoints. Esto es esperable dado que, a diferencia del resto, cada request implica dos llamadas secuenciales al servicio de transferencias, lo que mantiene conexiones abiertas por más tiempo y agota los recursos disponibles con mayor rapidez bajo alta concurrencia.



2.5. Components & Connectors

El sistema corre íntegramente en un único servidor orquestado con Docker Compose. El cliente se conecta vía HTTP al puerto 5555, donde Nginx actúa como reverse proxy redirigiendo cada request a la única instancia de la API Node.js en el puerto 3000, sin balanceo de carga ni redundancia. La persistencia del estado se realiza sobre archivos .json en disco local, sin base de datos externa ni almacenamiento compartido.



2. Métricas de Negocio

Para cumplir con este requerimiento, se instrumentó el endpoint POST /exchange para emitir métricas de negocio hacia StatsD inmediatamente después de cada operación de cambio exitosa. La implementación utiliza el módulo dgram de Node.js para enviar datagramas UDP al servicio StatsD (contenedor Graphite), sin necesidad de incorporar dependencias adicionales.

Por cada exchange completado con éxito se emiten cuatro contadores:

- `business.volume.{baseCurrency}`: incrementa en `baseAmount` y representa el volumen bruto operado en la moneda base.
- `business.volume.{counterCurrency}`: incrementa en `counterAmount` y representa el volumen bruto operado en la moneda de contrapartida.
- `business.net.{baseCurrency}`: incrementa en `baseAmount` y representa el neto desde la perspectiva de arVault, indicando que el sistema recibe esa moneda.
- `business.net.{counterCurrency}`: incrementa en `-counterAmount` y representa el neto desde la perspectiva de arVault, indicando que el sistema entrega esa moneda.

Los valores se acumulan en Graphite mediante la función `integral()` aplicada sobre los contadores `stats_counts.business.*`, lo que permite reconstruir la evolución acumulada del volumen y del neto a lo largo del tiempo. A partir de estas métricas se agregaron dos paneles al dashboard de Grafana.



El panel "Business — Gross volume by currency" muestra el volumen bruto acumulado por moneda. Durante la prueba de carga sobre el endpoint `/exchange` (escenario con patrón de trading diario, con un pico de 15 requests por segundo), el volumen de ARS alcanzó aproximadamente 15.800 unidades, mientras que el de USD fue cercano a 11,4 unidades, reflejando la diferencia nominal entre ambas monedas según el tipo de cambio configurado.

El panel "Business — Net by currency" muestra el neto acumulado por moneda desde la perspectiva de arVault. Se observa que el ARS presenta un valor neto positivo (~15.800), dado que el sistema recibe pesos en cada operación, mientras que el USD presenta un valor negativo (~ -11,4), ya que el sistema entrega dólares. Este comportamiento es consistente con el modelo de negocio implementado, donde arVault actúa como contraparte directa en las operaciones de cambio.

3. Tácticas Aplicadas

En esta sección se presentan las modificaciones propuestas e implementadas sobre el sistema, basadas en la aplicación de distintas tácticas de diseño orientadas a mejorar atributos de calidad. Para aquellos atributos que resultan mensurables, se incluyen métricas y evidencia empírica que permiten evaluar el impacto de los cambios realizados. En los casos donde no es posible una medición directa, se desarrolla una justificación teórica que sustenta las decisiones tomadas.

Asimismo, se analizan los resultados obtenidos, incluyendo situaciones en las que las mejoras no generan diferencias significativas o incluso afectan negativamente otros atributos, discutiendo sus posibles causas. Se aborda también el análisis de trade-offs entre distintos atributos de calidad, en función de las decisiones de diseño implementadas en el trabajo práctico.

Finalmente, para cada modificación que implique un cambio en la arquitectura, se presentan los correspondientes diagramas de Components & Connectors, con el objetivo de reflejar de manera clara la estructura y relaciones del sistema resultante.

4.1. Táctica 1: Transactions

El objetivo de esta táctica es mejorar el atributo de calidad Availability mediante la aplicación del concepto de **Transactions**, buscando que la operación de intercambio (exchange) se comporte de manera atómica y no deje al sistema en estados inconsistentes ante concurrencia o fallos parciales.

En la implementación original, la función exchange() no era atómica. El flujo consistía en validar el balance disponible, ejecutar operaciones asincrónicas (transfer) y luego actualizar los balances. Debido a la presencia de await, existían puntos donde la ejecución podía ser suspendida y otras requests concurrentes podían acceder y modificar el estado compartido antes de que la operación original finalizara. Esto generaba una condición de carrera en la cual múltiples operaciones podían validar el mismo balance y consumir los mismos fondos, produciendo inconsistencias como saldos negativos. Este tipo de errores no solo afecta la integridad de los datos, sino que también impacta directamente en la disponibilidad, ya que un sistema con datos corruptos puede fallar o quedar incapacitado para procesar nuevas operaciones.

Para resolver este problema, se implementó una estrategia basada en el patrón de reservation + rollback, que aproxima el comportamiento de una transacción a nivel de aplicación. La modificación clave consiste en agrupar el chequeo de balance y su actualización en un único bloque sincrónico, sin puntos de espera (await). De esta forma, aprovechando el modelo de ejecución single-thread de Node.js, se garantiza que ninguna otra request pueda interrumpir ese segmento crítico. En la práctica, esto implica que el monto a intercambiar se descuenta

inmediatamente del balance (reserva de fondos) antes de realizar cualquier operación externa.

Una vez realizada la reserva, se ejecutan las transferencias necesarias. Si todas las operaciones se completan correctamente, el estado final se consolida, completando el intercambio. En caso de que alguna operación falle, se activa un mecanismo de rollback que revierte los efectos parciales: se restauran los balances y, si corresponde, se deshacen transferencias previamente realizadas. De esta manera, la operación se comporta como “todo o nada” en la mayoría de los escenarios, evitando estados intermedios inválidos.

El resultado de esta implementación es la eliminación de condiciones de carrera sobre el balance y una mejora significativa en la consistencia del estado interno. Bajo pruebas de carga con múltiples requests concurrentes, el sistema deja de presentar inconsistencias como balances negativos y mantiene un comportamiento estable. Esto impacta positivamente en la disponibilidad, ya que reduce la probabilidad de fallos en tiempo de ejecución y evita situaciones donde el sistema no pueda continuar operando debido a datos corruptos.

Sin embargo, es importante destacar que esta solución no constituye una transacción completa en el sentido estricto de las propiedades ACID. Se trata de una aproximación a nivel de aplicación que garantiza atomicidad sobre el estado interno, pero no provee aislamiento en entornos distribuidos ni durabilidad persistente. Además, existe un caso límite en el que la atomicidad no se cumple completamente: si una transferencia falla y también falla su operación de compensación (rollback), puede producirse un estado parcialmente ejecutado. Resolver este tipo de escenarios requiere mecanismos más avanzados, como el uso de patrones de tipo Saga o protocolos de two-phase commit.

La **disponibilidad** es el atributo más beneficiado: al evitar estados intermedios inválidos, se reduce la probabilidad de fallos en tiempo de ejecución y se evitan situaciones donde el sistema no pueda continuar operando debido a datos corruptos. La integridad de los datos mejora en igual medida, ya que bajo carga concurrente el sistema deja de presentar balances negativos y mantiene un comportamiento estable. Ambos atributos están estrechamente vinculados a la **fiabilidad** del sistema, que en el contexto de una fintech resulta quizás el atributo más crítico de todos: un servicio que opera con datos financieros inconsistentes no solo falla técnicamente, sino que compromete la confianza del usuario en la plataforma, que es el activo más difícil de recuperar una vez perdido.

En cuanto a la **mantenibilidad**, la solución introduce una mayor complejidad en la lógica de negocio debido al manejo explícito de rollback y control de errores, lo que eleva la carga cognitiva para quien deba modificar o extender el código en el futuro.

Respecto a la **performance**, puede existir un leve impacto al priorizar consistencia por sobre throughput, aunque este tradeoff es deliberado y deseable: en una fintech, garantizar que cada operación sea correcta tiene precedencia sobre maximizar la cantidad de operaciones procesadas por segundo.

En conclusión, la táctica implementada representa una solución adecuada al alcance del problema: sin incorporar dependencias externas ni mecanismos de coordinación distribuida, se logra un comportamiento sustancialmente más robusto frente a la concurrencia y fallos parciales. Las limitaciones identificadas respecto a las propiedades ACID completas son conocidas y aceptadas, y constituyen el punto de partida natural para evoluciones futuras de la arquitectura.

4.2. Táctica 2: Limit Access

Se implementó la táctica **Limit Access** mediante rate limiting por IP usando la librería express-rate-limit. El objetivo es proteger el servicio ante un volumen excesivo de requests, ya sea por abuso intencional o por picos de tráfico no anticipados, limitando la cantidad de operaciones que un mismo cliente puede realizar en una ventana de tiempo. De esta manera, cuando un cliente supera el límite configurado, el servidor responde con HTTP 429 (Too Many Requests)

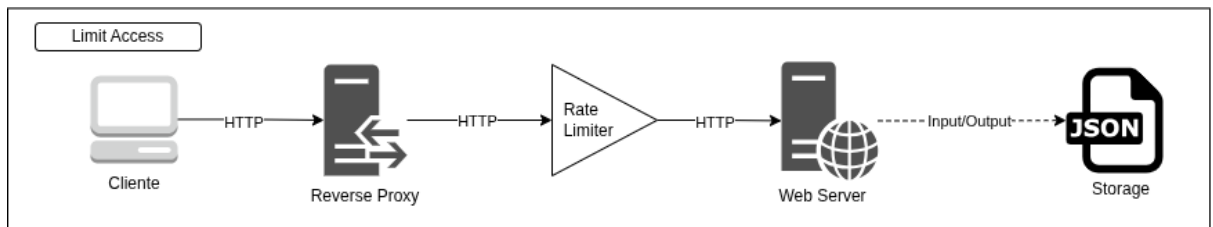
Se configuraron límites diferenciados por endpoint, priorizando la sensibilidad del dato por sobre el costo computacional, dado que en una fintech la integridad de la información es más crítica.

- **Límite global:** 60.000 req/min (~1.000 req/s), como primera línea de defensa sobre todos los endpoints.
- **GET /rates, GET /log y GET /accounts:** 36.000 req/min (~600 req/s), lecturas livianas sobre memoria sin modificación de estado, por lo que admiten una tasa más alta sin riesgo para la integridad del sistema.
- **POST /exchange:** 24.000 req/min (~400 req/s), al ser el core del negocio se busca un límite que proteja el sistema sin restringir el flujo normal de operaciones, dado que rechazar tráfico en este endpoint impacta directamente en los ingresos.
- **PUT /rates y PUT /accounts/:id/balance:** 12.000 req/min (~200 req/s), el límite más restrictivo a pesar de ser operaciones computacionalmente livianas. Una ráfaga de escrituras sobre tasas de cambio o balances representa un riesgo de integridad mayor que un volumen alto de operaciones de exchange (en el primer caso el daño recae sobre el estado compartido del sistema, mientras que en el segundo cada operación es atómica e independiente).

Cabe aclarar que estos valores están dimensionados para el escenario de prueba utilizado, donde todo el tráfico se origina desde una única IP. En un entorno productivo real, el tráfico estaría distribuido entre múltiples clientes, por lo que los

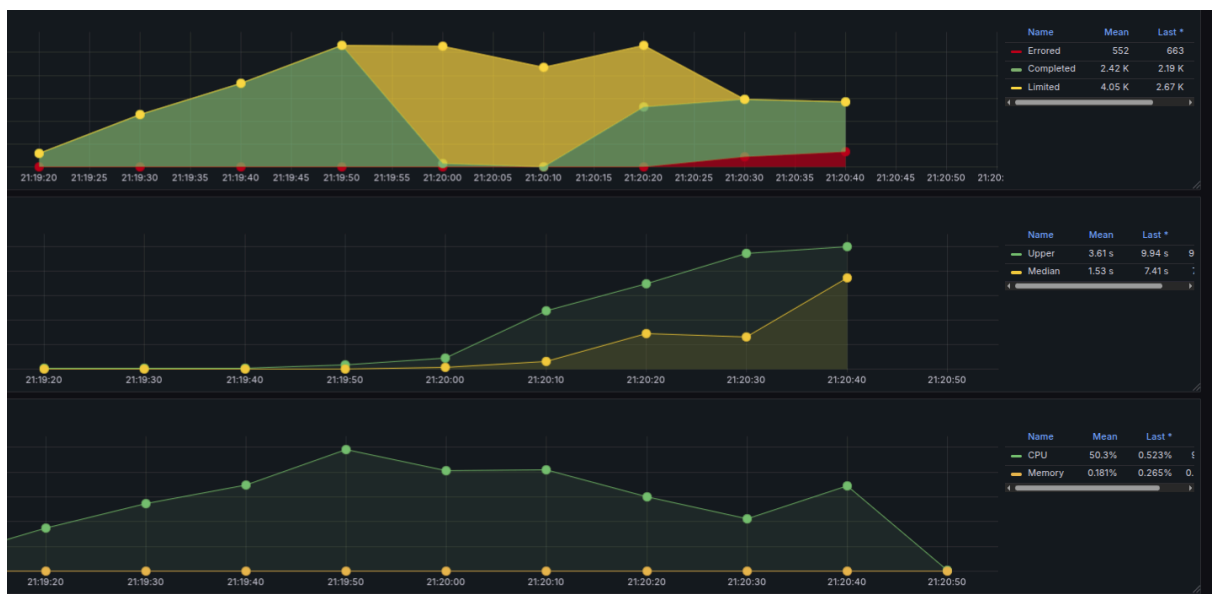
límites deberían reducirse considerablemente para cumplir su función de protección efectiva.

Tras la aplicación de la presente táctica, el nuevo diagrama de Components & Connectors queda definido como se observa a continuación:



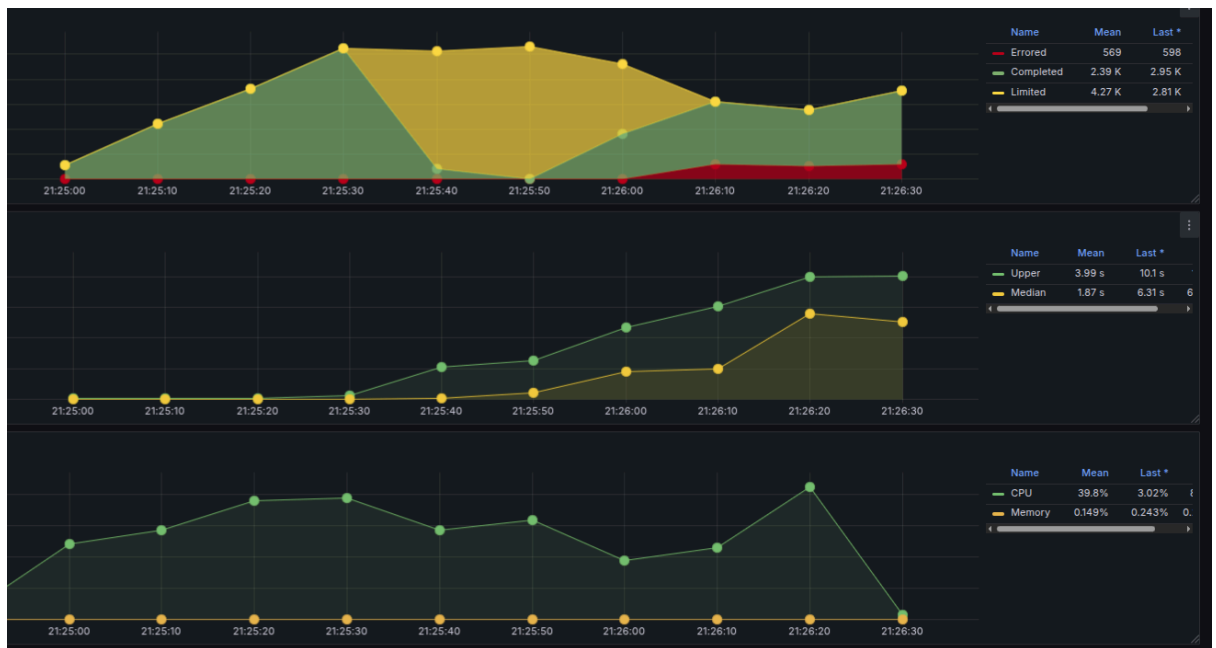
Para validar la táctica implementada, se ejecutó el escenario de estrés sobre ambos servicios de tipo PUT, dada su importancia a nivel negocio.

Al ejecutar el mismo escenario de estrés para PUT /rates , la prueba arrojó un CPU del 50,3% y memoria del 0,181%, este último valor notablemente inferior al pre rate limiting. De los 36.020 requests, 12.158 recibieron HTTP 429, 22.210 fueron exitosos y 1.652 fallaron (ECONNRESET: 1.399, ETIMEDOUT: 253). El efecto del rate limiting es visible: al rechazar anticipadamente una porción del tráfico, la tasa de fallos reales se redujo del 3,3% al 1,7% (excluyendo los 429, que son rechazos controlados y esperados). Los tiempos de respuesta de las solicitudes exitosas (2xx) muestran una mediana de apenas 15 ms, muy inferior a los 210,6 ms pre rate limiting, lo que indica que las requests que logran pasar el límite son procesadas con mayor fluidez al haber menos competencia por los recursos.



En segundo lugar, la prueba post rate limiting de PUT /accounts/:id/balance arrojó un CPU del 39,8% y memoria del 0,149%, ambos valores notablemente

inferiores al pre rate limiting, siendo la reducción de CPU la más significativa de los dos endpoints PUT analizados. De los 36.020 requisitos, 12.818 recibieron HTTP 429, 21.496 fueron exitosos y 1.706 fallaron (ECONNRESET: 1.197, ETIMEDOUT: 509). La tasa de fallos reales se redujo del 8,3% al 4,7%, la mejora más pronunciada observada entre ambos endpoints PUT. Al igual que en PUT /rates, las solicitudes que lograron pasar el límite fueron procesadas con mayor fluidez, reflejado en una mediana de tan solo 23,8 ms para las respuestas 2xx.



Pensando en los atributos de calidad, sobre la **seguridad** es posible mencionar que el rate limiting es una barrera concreta contra ataques de denegación de servicio (DoS) y fuerza bruta, limitando la capacidad de un actor malicioso de saturar el servicio o explorar combinaciones de parámetros a alta velocidad. Vale aclarar que la táctica es efectiva contra ataques simples pero no contra ataques coordinados sofisticados: un atacante distribuido que opere desde múltiples IPs puede evadir los límites por IP sin dificultad.

Al limitar la tasa de requests por IP, se evita que un cliente descontrolado o malicioso consuma todos los recursos del servidor y degrade el servicio para el resto de los usuarios, lo cual se traduce en una mejora en la **disponibilidad**. Esto se evidencia en los resultados: bajo el mismo escenario de estrés, el uso de CPU cayó y los requests que lograron procesarse lo hicieron con tiempos consistentes con el load test. Sin embargo, existe un tradeoff concreto: el rate limiter preserva la estabilidad del servidor, pero la disponibilidad percibida desde el cliente que recibe un 429 es nula. La táctica prioriza la estabilidad del servicio por sobre la atención de la totalidad del tráfico entrante.

La **performance** debe analizarse desde dos perspectivas. Para clientes que no superan el límite, el impacto es despreciable dado que el middleware agrega latencia

mínima: las requests admitidas se procesan con mayor fluidez al haber menos competencia por los recursos, lo que se refleja en medianas de respuesta de apenas 15 ms en PUT /rates y 23,8 ms en PUT /accounts, frente a los 210,6 ms y 273,2 ms registrados sin limitación bajo el mismo escenario de estrés. Para clientes que sí lo superan, la experiencia empeora abruptamente: reciben un 429 en lugar de ser encolados o atendidos con mayor latencia.

A su vez, la **modificabilidad** se ve levemente afectada dado que los valores de max y windowMs están hardcoded en el código, lo que implica que cualquier ajuste requiere un redespigüe. Definirlos como variables de entorno mitiga parcialmente el problema pero no lo resuelve, ya que igualmente requiere reiniciar el proceso para que los nuevos valores tomen efecto.

Finalmente, el rate limiting en memoria presenta un tradeoff importante a futuro en cuanto a la **escalabilidad horizontal**. Si se despliegan múltiples instancias de la API, cada una mantiene sus propios contadores independientes, por lo que el límite efectivo se multiplica por la cantidad de instancias y pierde sentido. La solución natural es centralizar los contadores en un store compartido como Redis.

En conclusión, la implementación del rate limiting representa una mejora concreta en la resiliencia del sistema frente a escenarios de abuso o tráfico extremo, cambiando el modo de falla de colapso no controlado a rechazo explícito y predecible. Sin embargo, su efectividad está acotada: no elimina todos los errores bajo carga extrema, no protege contra ataques distribuidos, y presenta limitaciones de escalabilidad horizontal en su implementación actual.

4.3. Táctica 3, 4 y 5: Distributed State Cache con Scale out not up e Introduce Concurrency

El objetivo de esta táctica es habilitar la **escalabilidad horizontal** del sistema mediante la externalización del estado a un **caché distribuido**, eliminando la dependencia del estado en memoria del proceso y resolviendo simultáneamente las deficiencias de persistencia identificadas en el análisis inicial.

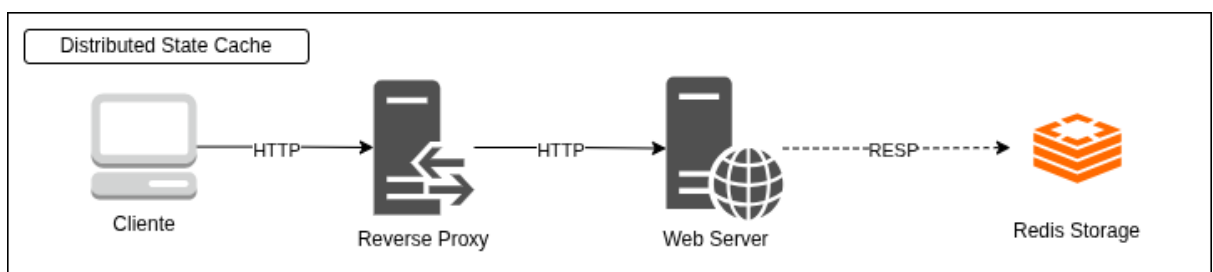
En la implementación original, todo el estado del sistema (cuentas, tasas y log de transacciones) vivía exclusivamente en la memoria RAM del proceso Node.js, sincronizándose de forma periódica con archivos .json locales en disco. Este diseño hacía arquitectónicamente imposible el escalado horizontal: si se levantaran múltiples instancias, cada una mantendría su propia copia independiente del estado en memoria. Las modificaciones realizadas en una instancia no serían visibles para las demás, y todas sobrescribirían los mismos archivos en disco generando corrupción de datos. Adicionalmente, la ventana de pérdida de datos ante un crash podía alcanzar hasta 5 segundos, y las escrituras no eran atómicas.

Para resolver este problema se implementó Redis como store de estado compartido entre instancias, aplicando la táctica de **Distributed State Cache**. La implementación se concentra en el módulo `persistence.js`, que reemplaza completamente la lógica de acceso a archivos JSON por operaciones sobre Redis. Las cuentas y tasas de cambio se almacenan como strings JSON serializados (vía GET/SET), mientras que el log de transacciones se implementa como una lista Redis (vía RPush/LRange), garantizando orden de inserción sin necesidad de deserializar y rearmar el array completo.

El aspecto más crítico de la implementación es la operación de transferencia de fondos entre cuentas internas. En el sistema original, la verificación del saldo y su actualización eran operaciones separadas en memoria, lo que exponía una ventana de condición de carrera cuando múltiples instancias intentaban operar sobre las mismas cuentas simultáneamente. Para resolverlo, se implementó *atomicFundsTransfer* mediante un script Lua evaluado atómicamente en el servidor Redis (comando EVAL). Este script realiza, en una única operación indivisible y bajo exclusión mutua garantizada por el modelo de ejecución single-threaded de Redis: la lectura del estado de cuentas, la verificación del saldo disponible en la moneda de contrapartida y la actualización simultánea de ambos balances. Dado que Redis garantiza que ningún otro comando puede interrumpir la ejecución de un script Lua, se elimina por completo la condición de carrera entre instancias concurrentes.

A nivel de infraestructura, se construyó una imagen Docker personalizada para Redis que incorpora: el arranque con persistencia AOF (`--appendonly yes`) para durabilidad del estado ante reinicios, y un script de inicialización (`entrypoint.sh`) que siembra los datos iniciales de cuentas y tasas en el primer arranque y los omite en los siguientes, preservando el estado existente. En el `docker-compose.yml` se agregó un health check mediante `redis-cli ping`, y la API recibe la URL de conexión vía variable de entorno `REDIS_URL`, lo que permite apuntar a distintas instancias de Redis en diferentes entornos.

A continuación, se observa el nuevo diagrama de Componentes & Connectors tras la inclusión de Redis:



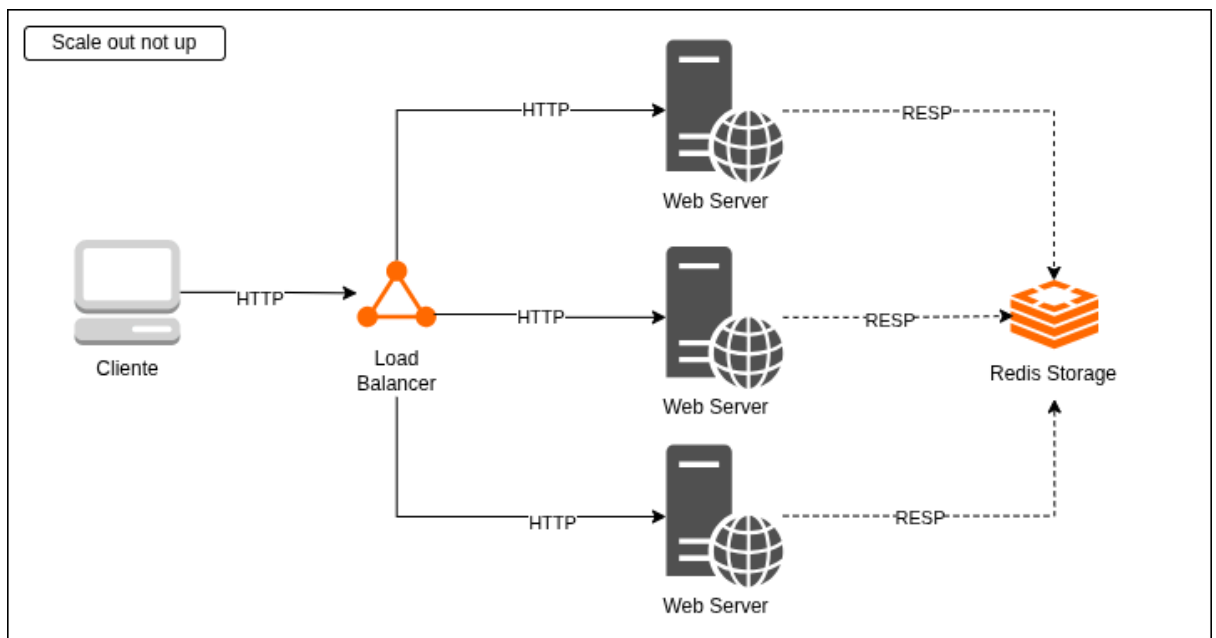
Con el estado externalizado a Redis, el **escalado** horizontal se vuelve trivial: todas las réplicas comparten el mismo estado coherente y cualquier instancia puede atender cualquier request sin inconsistencias. Se desplegaron tres réplicas del

servicio Node.js declarando réplicas: 3 en docker-compose.yml, todas conectadas al mismo nodo Redis vía variable de entorno REDIS_URL.

Nginx actúa como único punto de entrada externo, distribuyendo el tráfico entre las réplicas con el algoritmo round-robin, que deriva las requests de forma equitativa y rotativa entre las instancias disponibles. Adicionalmente, se habilita keep-alive entre Nginx y las réplicas, reutilizando conexiones TCP y reduciendo la latencia de establecimiento de conexión bajo alta carga.

Además, las réplicas no exponen puertos directamente al exterior: el puerto 5555 del host mapea únicamente al puerto 80 de Nginx, que es el único punto de entrada al sistema.

Para esta nueva etapa de la implementación, se puede observar el siguiente diagrama:



Finalmente, se aplica la táctica de **Introduce Concurrency**, la cual se implementa sobre la arquitectura ya descrita. En ese contexto, se identificó que POST /exchange continuaba siendo el cuello de botella principal del sistema.

En su implementación original, el flujo exitoso de una operación de cambio encadenaba dos llamadas a transfer() con await secuencial, cada una con una latencia simulada de entre 200 y 400 ms. Esto dejaba el tiempo de respuesta del happy path entre 400 y 800 ms aún antes de considerar overhead de routing, parseo o acceso a estado, lo que en la práctica significaba que cada operación de cambio tardaba casi un segundo en confirmarse.

La táctica aplicada es **Introduce Concurrency** en su variante intra-request: paralelizar operaciones lógicamente independientes que se ejecutaban en serie sin necesidad. Se identificaron dos oportunidades concretas de paralelización en el flujo de `exchange()`:

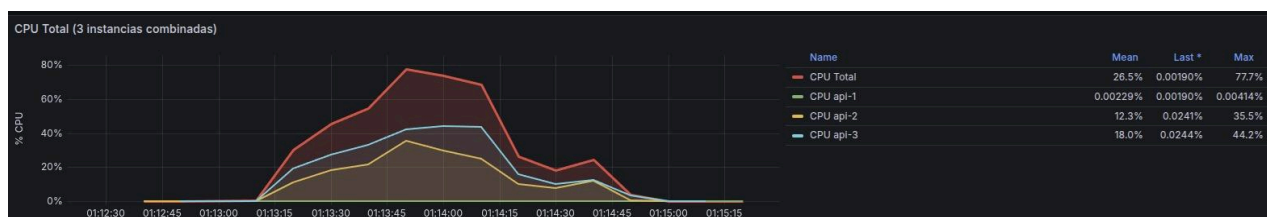
La primera corresponde a la lectura inicial de estado desde Redis. En la versión con estado externalizado, las consultas de tasas de cambio y cuentas se realizaban secuencialmente, esperando la respuesta de Redis para cada una antes de lanzar la siguiente. Dado que ambas lecturas son completamente independientes entre sí, se reemplazaron por una única llamada `Promise.all()`, lanzando ambas consultas en paralelo y esperando a que ambas se completen. El tiempo total pasa de ser la suma de ambas latencias de red a ser el máximo de ellas.

La segunda y más significativa oportunidad corresponde a las dos transferencias: la que retira los fondos del cliente en la moneda base y la que entrega la moneda counter desde la cuenta interna. Estas operaciones no comparten dependencia de datos entre sí, por lo que serializar su ejecución era un desperdicio directo de tiempo. Al ejecutarlas en paralelo mediante `Promise.all()`, el tiempo total de este bloque se reduce del rango 400–800 ms (suma de ambas transferencias) a aproximadamente 200–400 ms (máximo de ambas). Esta optimización es especialmente efectiva dado que `transfer()` es una operación puramente I/O-bound: el costo real es esperar y no consumir CPU, por lo que las dos esperas pueden superponerse sin penalización adicional sobre el event loop de Node.js.

Tras la aplicación del conjunto de tácticas (Redis como estado compartido, escalado a tres réplicas y paralelización de transferencias), los resultados muestran una mejora sustancial respecto al caso base. El CPU descendió del 33,7% al 12,3% en la instancia 2, 18% en la instancia 3 y al 0,00229% en la primera instancia. Esto refleja el efecto de distribuir la carga entre réplicas y reducir el tiempo que cada request mantiene recursos ocupados. Además, la memoria pasó del 0,512% al 0,135% en promedio entre las tres instancias,

De los 36.020 requests, 31.874 fueron exitosos y 4.146 fallaron por `ECONNRESET`, representando una tasa de fallo del 11,5%, levemente superior al caso base. Sin embargo, la naturaleza de los errores cambia significativamente desaparecen por completo los errores HTTP 502, que en el caso base indicaban que Nginx no podía establecer conexión con Node.js. Todos los fallos son ahora `ECONNRESET`, lo que sugiere saturación de conexiones bajo el pico extremo de carga pero sin interrupciones totales del servicio.

La mejora más notable se observa en los tiempos de respuesta. La media cayó de 1.153,6 ms a 343,3 ms y la mediana de 713,5 ms a 354,3 ms, valor consistente con el tiempo esperado de dos transferencias paralelas. El p95 se redujo de 5.272,4 ms a 407,5 ms y el p99 de 7.117 ms a 432,7 ms, evidenciando que incluso en los percentiles altos el sistema se comporta de forma mucho más predecible y controlada que en el caso base.



El atributo más favorecido por el conjunto de tácticas aplicadas es la **escalabilidad**. El sistema pasa de una arquitectura donde el escalado horizontal era imposible a una donde agregar réplicas se reduce a modificar el valor de réplicas en `docker-compose.yml`. El límite de escalado queda determinado por la capacidad de Redis y de Nginx, y no por la lógica de la aplicación.

La **performance** se ve beneficiada desde dos ángulos complementarios. Por un lado, la distribución de carga entre tres réplicas aumenta el throughput global del sistema, compensando la latencia de red adicional que introduce Redis respecto al acceso en memoria. Por otro, la paralelización de las transferencias dentro de cada request reduce el tiempo de respuesta de POST /exchange de hasta 800 ms a aproximadamente 600 ms, mejora directamente observable y relevante en el contexto de una fintech donde la velocidad de confirmación de una operación forma parte de la propuesta de valor.

Respecto a la **availability**, la externalización del estado elimina la pérdida de datos ante el crash de una instancia Node.js, ya que el estado persiste independientemente del proceso. La persistencia AOF reduce además la ventana de pérdida ante reinicios de Redis. No obstante, Redis introduce un nuevo único punto de falla: si cae, las tres réplicas quedan inoperativas simultáneamente, limitación que podría mitigarse en el futuro con Redis Sentinel o Redis Cluster.

La **fiabilidad** e integridad de los datos mejoran significativamente. El script Lua atómico elimina la condición de carrera identificada en el sistema original, garantizando que las operaciones de exchange sean indivisibles incluso bajo alta concurrencia entre réplicas, resolviendo por completo el riesgo de saldos negativos que existía en la implementación original.

En cuanto a la **mantenibilidad**, las tácticas introducen un tradeoff acumulado. La separación entre lógica y persistencia en `persistence.js` mejora la cohesión del código y facilita el despliegue en distintos entornos, contribuyendo también a la testeabilidad. Sin embargo, el código concurrente basado en `Promise.all()` eleva la carga cognitiva para quien deba modificarlo, y la arquitectura en su conjunto es considerablemente más compleja de operar: requiere gestionar Redis con su propia configuración, persistencia y disponibilidad.

Finalmente, la **seguridad** presenta un nuevo frente de atención. Redis centraliza el estado completo del sistema y su exposición sin autenticación representaría una superficie de ataque crítica. En la implementación actual esto es

aceptable dado que opera únicamente dentro de la red interna de Docker, pero en un entorno productivo sería imprescindible habilitar autenticación mediante requirepass y restringir su acceso explícitamente.

En conclusión, el conjunto de tácticas aplicadas representa la modificación arquitectónica más profunda del sistema: se pasa de una arquitectura monolítica con estado en memoria a una distribuida, horizontalmente escalable y con garantías de atomicidad en las operaciones financieras críticas. Los beneficios son significativos y directamente observables, pero vienen acompañados de mayor complejidad operativa, nuevos riesgos de seguridad y la introducción de Redis como punto de falla centralizado, tradeoffs que deben considerarse en la evolución futura de la arquitectura.

4. Conclusiones

El análisis de arVault partió de un sistema que, si bien cumplía su función básica, presentaba limitaciones arquitectónicas que lo hacían inadecuado para operar en condiciones de carga real. El estado compartido en memoria, las transferencias secuenciales, la ausencia de control de concurrencia y la inexistencia de mecanismos de protección ante tráfico excesivo constituían un conjunto de problemas estructurales que se potenciaban entre sí.

Las pruebas de carga confirmaron que los endpoints respondían en tiempos razonables a baja concurrencia. Sin embargo, bajo escenarios de estrés el sistema se degradaba de forma severa, alcanzando tasas de fallo de entre 6% y 10%, latencias p99 superiores a 9 segundos y errores HTTP 502 que evidenciaban interrupciones completas del servicio en los picos de mayor demanda.

La táctica de mayor impacto arquitectónico fue la combinación de Distributed State Cache con Scale Out Not Up e Introduce Concurrency. Externalizar el estado a Redis funcionó como condición necesaria para habilitar el escalado horizontal. Sin esta modificación, agregar réplicas no solo dejaba intacto el problema original, sino que además introducía inconsistencias por escrituras concurrentes sobre estados mantenidos en memoria local.

Uno de los aspectos más delicados de esta táctica fue la implementación del script Lua atómico para las transferencias de fondos. El desafío consistía en eliminar condiciones de carrera entre múltiples instancias concurrentes, algo que el modelo de ejecución de Node.js no puede garantizar por sí solo en un entorno distribuido. Con Redis actuando como árbitro del estado compartido, el sistema pasó de una arquitectura donde el escalado horizontal era inviable a otra donde incorporar nuevas instancias depende únicamente de configuración.

Además, al incluir también la concurrencia, se pudo observar la mejora de performance de forma más directa. La paralelización de las dos transferencias ejecutadas por el endpoint POST /exchange redujo la mediana de respuesta de 713,5 ms a 354,3 ms y disminuyó el p95 de 5.272,4 ms a 407,5 ms, todo ello sin modificar la arquitectura general ni incorporar dependencias adicionales.

Desde el punto de vista del negocio, esta mejora tiene implicancias directas. Una fintech cuyo diferencial es ofrecer mejores tasas de cambio también compite en velocidad de ejecución. Cuando un usuario obtiene una cotización favorable, una espera cercana al segundo introduce incertidumbre respecto de la operación. Reducir la latencia disminuye esa percepción de riesgo y además libera recursos del servidor, permitiendo sostener un mayor volumen de operaciones concurrentes sobre la misma infraestructura.

La táctica de Transactions estuvo orientada al atributo probablemente más crítico de un sistema financiero: la fiabilidad de los datos. Implementar el patrón de reservation + rollback permitió eliminar condiciones de carrera sobre los balances aprovechando el modelo single-thread de Node.js para ejecutar el chequeo de saldo y el descuento como una operación indivisible.

La táctica de Limit Access modificó el comportamiento del sistema frente al tráfico excesivo. Antes de su implementación, la sobrecarga derivaba en fallos de conexión y colapsos no controlados. Luego del agregado del rate limiter, el sistema pasó a rechazar solicitudes de manera explícita mediante respuestas HTTP 429.

Los resultados reflejaron una mejora concreta en estabilidad. Bajo el mismo escenario de estrés, el consumo de CPU del endpoint PUT /accounts descendió del 52% al 39%, mientras que la tasa de fallos reales se redujo del 8,3% al 4,7%.

Esta táctica también expone uno de los trade-offs más evidentes del trabajo. El servidor preserva su estabilidad a costa de rechazar tráfico legítimo, por lo que la disponibilidad percibida por el cliente afectado es efectivamente nula. Además, la implementación actual del rate limiter utiliza almacenamiento en memoria, lo que la vuelve incompatible con el escalado horizontal introducido previamente. Cada réplica mantiene contadores independientes y el límite efectivo aumenta junto con la cantidad de instancias desplegadas.

Centralizar esos contadores en Redis aparece como la evolución natural de la solución, ya que permitiría mantener coherencia entre réplicas y extender de forma consistente la arquitectura distribuida ya implementada.

El análisis transversal de las tácticas muestra un patrón recurrente. Cada mejora sobre un atributo introduce nuevas complejidades sobre otros aspectos del sistema. Redis mejora la escalabilidad, aunque agrega un nuevo punto de falla centralizado y aumenta la complejidad operativa. La concurrencia reduce la latencia,

pero vuelve más difícil el manejo de errores. Las transacciones incrementan la fiabilidad, aunque dificultan la testeabilidad. El rate limiting protege la disponibilidad del servidor, pero degrada la experiencia del cliente que excede el límite permitido.

Lejos de representar un error de diseño, este comportamiento refleja los trade-offs inherentes a cualquier arquitectura de software. No existe una táctica capaz de optimizar simultáneamente todos los atributos de calidad; cada decisión debe alinearse con las prioridades del negocio y aceptar explícitamente los costos asociados.

En el caso de arVault, dichas prioridades son claras. Un servicio financiero que busca captar inversores necesita priorizar, antes que nada, la fiabilidad de los datos, seguida por la disponibilidad y finalmente por una performance capaz de sostener su propuesta de valor.

Las tácticas implementadas avanzan de forma consistente en esa dirección. El sistema resultante es más robusto, más escalable y más seguro que el recibido originalmente. Aun así, persisten áreas de mejora claramente identificadas, entre ellas la ausencia de alta disponibilidad en Redis, la necesidad de centralizar los contadores del rate limiter y la incorporación futura de mecanismos como Saga o two-phase commit para aproximarse a propiedades ACID completas en operaciones distribuidas.

Estas limitaciones conocidas constituyen el punto de partida natural para la evolución futura de la arquitectura.